LECTURE 2

Introduction to Inductive Synthesis

One of the simplest interfaces for program synthesis is *Inductive synthesis*. In inductive synthesis, the goal is to generate a function that matches a given set of input/output examples. The literature makes a distinction between *Programming by Example (PBE)*, and *Programming by Demonstration (PBD)*. In Programming by example, the goal is to infer a function given only a set of inputs and outputs, whereas in Programming by Demonstration, the user also provides a trace of how the output was computed.

For example, in Programming-by-example, if I want to convey to the system that I want it to synthesize the factorial function I may give it an example:

factorial$(6) = 720$

As one can see, this is a highly under-specified problem, since there is an enormous number of possible functions that may return 720 given the input 6. By contrast, with programming-by-demonstration, one may provide a more detailed trace of the computation:

factorial$(6) = 6 * (5 * (4 * (3 * (2 * 1)))) = 720$

In general, the full trace in programming-by-demonstration contains more information that makes it easier to infer the intended computation. The line between PBD and PBE can be blurry, however. For some domains (like string manipulation), it's relatively easy to derive a trace from an output, and different systems may capture the trace at different levels of fidelity.

2.1 History

The idea of directing a computer through examples dates back to the 1970s, when Patrick Winston at MIT published the seminal work "Learning Structural Descriptions from Examples" [1]. This work was among the first to look into the problem of generalizing from a set of observations, although it was not really about trying to automate programming.

A good candidate for "first PBD system" is Pygmalion [2]. Pygmalion was framed as an "interactive 'remembering' editor for iconic data structures"; the high-level idea was that a program would move icons around and establish relationships between them, but the user would not write the program itself; instead, the user would manipulate the icons directly and the editor would remember the manipulations and be able to apply them in other contexts. This was explained through their "Basic pygmalion metaphor":

a program is a series of EDITING CHANGES to a DISPLAY DOCUMENT.

Input to a program is an initial display document, i.e. a display screen containing images. Programming consists of editing the document. The result of a computation is a modified document containing the desired information. (*emphasis in the original*) [2]

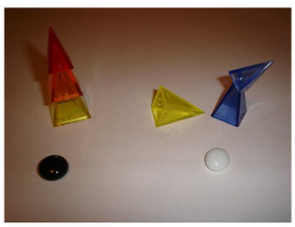
The idea of a "remembering editor" was that if you perform such a manipulation by hand once, the editor can remember how to perform the manipulation so you can apply a similar manipulation in other contexts. Like a lot of early AI work, PYGMALION was very heavy on philosophy and metaphor and very weak on algorithms, particularly around the crucial question of how to generalize from a given demonstration so that the learned program could apply in other situations.

At around the same time, Summers looked more systematically at the question of how to generalize from a demonstration to a program, particularly at the question of how to derive looping structure [3] [4]. His algorithm was based on pattern matching, and was relatively brittle, but is still considered an important algorithm in the field.

A little bit of history: who was the first?

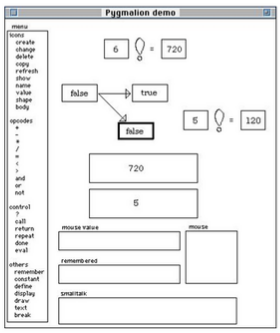
Learning Structural Descriptions from Examples

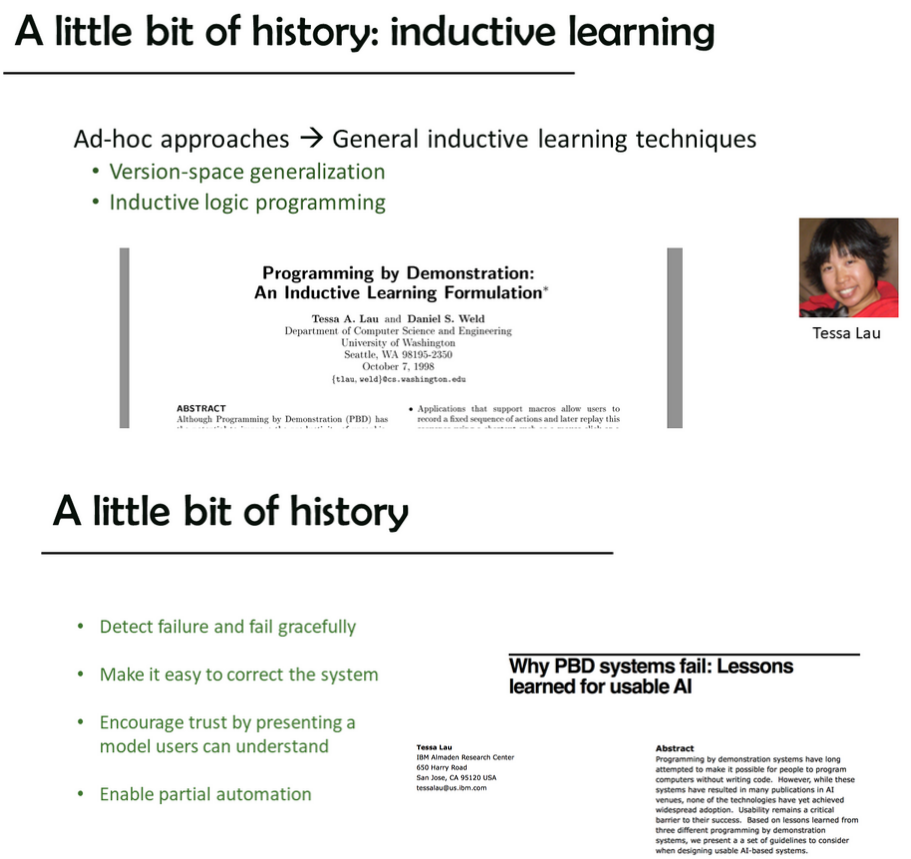
- By our very own Patrick Winston [1970]
- Explored the question of generalizing from a set of observations
- Similar to the Zendo game:



Pygmalion [Smith 75]

- Real programming by demonstration
- http://acypher.com/wwid/Chapters/01Pygmalion.html



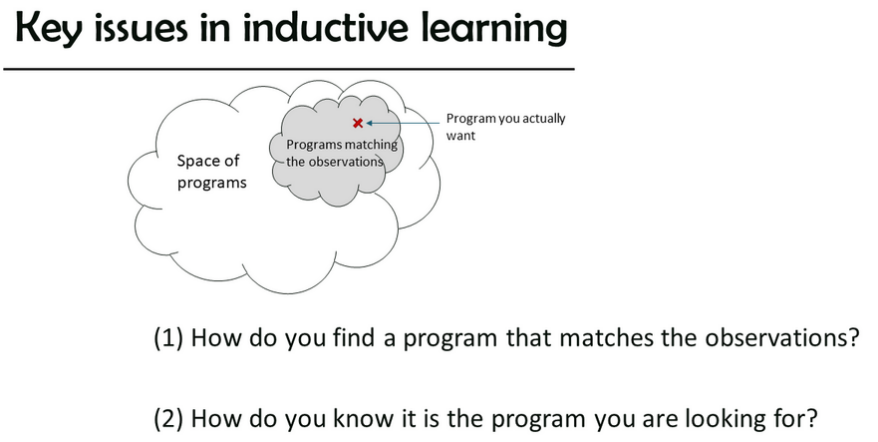


Over time, much of the interest from the AI community shifted to Machine Learning, and to approaches to infer functions from large amounts of noisy data instead of a small number of careful demonstrations, with very little progress being made in the PBD and PBE space. There was another burst of interest in the mid 1990s that is best exemplified by the work of Tessa Lau, which tried to bring back insights from machine learning into PBE/PBD. Tessa Lau started this work as a graduate student at UW while working with Daniel Weld and she continued as a researcher at IBM [5]. The goal was to move away from ad-hoc and brittle approaches and to develop general techniques that could be adapted to a variety of PBE problems.

The work focused on two major techniques: Version space generalization (which will be discussed later) and Inductive logic programming (which is beyond the scope of this course). This line of work caused a lot of excitement for a while, but it petered out after it was clear that they were not solving the problem well enough to be practical. Interestingly, only a couple of years before FlashFill launched the modern wave of programming-by-example systems, Tessa Lau published an article titled "Why PBD systems fail: Lessons learned

for usable AI" [6], which articulated many of the pitfalls that prevented the success of PBD systems.

2.2 Framing the PBD/PBE Problem



There are two core challenges in the PBE/PBD paradigm. The first challenge is *How do you find a program that matches the observations?* Where the observations can be either input/output examples or richer execution traces as in PBD. The second challenge is *How do you know the program you found is the one you were actually looking for?* At the end of the day, both PBD and PBE are fundametally under-specified problems, and there is a potentially large space of possible programs that match the given observations, so how do we know which one is the one the user actually wants?

In traditional machine learning, the focus has historically been on the second challenge. The trick has been to pick spaces of programs that are either: extremely expressive (e.g. neural networks), so that there are many different ways to match any set of observations and the challenge reduces to avoiding over-training; or too restricted (e.g. SVMs or Gaussian Linear Models), such that it is more or less impossible to match all the observations, but you assume some observations are wrong anyway, so you can trade off how many samples you match against other criteria that make it more likely that your solution will work well enough in general.

The modern emphasis in PBE, however, has been to focus more on the space of programs itself. The focus on restricting the space of programs is not new; many early systems did this to an extreme degree by just having a short list of programs that the system would scan through looking for one that matched the examples. What recent advances in synthesis have brought to the table are powerful mechanisms to search *arbitrary* program spaces. This has allowed us

to design the space of programs in a way that excludes undesirable solutions from the space and focuses the search on "reasonable" programs. The ability to carefully control the program space does not completely eliminate the need to rank programs to give priority to the most likely ones. Even with a carefully designed program space, the problem is still underspecified. But the idea is that if you can search large but highly constrained space efficiently, you are more likely to get what you are looking for. By having the ability to reason about arbitrary (and very large) spaces of programs you can get the benefits of the list-of-programs approach without its inherent brittleness.

2.3 What is a program?

This is a good point to consider the question of what we actually mean when we talk about a *program* in the context of program synthesis. A program is a description of how to perform a computation. In general, describing a program requires a notation, a *programming language* that allows you to describe many different computations by composing individual syntactic elements each with well defined meaning. We are all familiar with popular programming languages such as `Pyton`, `JavaScript` or `C`. At the other extreme, the notation of arithmetic, for example, can also be considered a programming language; it includes syntactic elements such as numbers and arithmetic operators ($+$, $-$, $\times$), each with a well defined meaning which can be used to describe a particular kind of computation. Unlike a language like `Python`, the language of arithmetic is not universal; it can only be used to described a very narrow class of computations, so we can use it to compute the tip in a restaurant bill, but not to determine the smallest element in a list of numbers.

As we will see over the rest of this course, the choice of an appropriate notation is crucial to the success of program synthesis. Even if your end goal is to generate code in `Pyton` or `C`, it is often useful to frame the program synthesis problem in terms of a more narrow notation that more precisely captures only those programs that are actually relevant to the task at hand. We will often refer to these more specialized langauges as *Domain Specific Languages* (DSLs). It is usually convenient to think of them simply as subsets of more general languages, where the programmer is prevented from using particular constructs, and is provided only with a limited set of functions or subroutines.

Throughout the rest of this course, we will be defining small DSLs as synthesis targets. Often, it will be enough to define their semantics informally or through examples, or where more precision is warranted, we will define it in terms of a general purpose programming language. In many settings, we will be using the notation of functional programming, which will be familiar to anyone who has programmed in `Haskell` or `Ocaml`, but may seem a bit foreign to some. We will say more about this notation when we use it, but at a high-level,

these are a few things to keep in mind about this notation, and why it makes a good target for synthesis:

- **No side effects.** Computation in a functional language happens by evaluating pure functions, with no side effects and no mutation. This can be annoying when programming by hand, but can often simplify the job of the program synthesizer. A consequence of this is that functions are actually pure functions, if you give them the same inputs, they will produce the same outputs. This can also simplify the reasoning process significantly.
- **Concise and expressive.** Think about a program in Java that reverses a list. It's a relatively long program that involves a class declaration, some method declarations, some loops, some constructors; maybe it would look something like this:

```
class ListReverser{
  static List reverseList(List myList) {
    List output = new ArrayList();
    for (int i = 0; i < myList.size(); i++) {
        output.add(myList.get(myList.size() - 1 - i));
    }
    return output;
  }
}
```

  You can probably write code that is slightly cleaner than the code above, but not by much. From a synthesis perspective, that is a lot of code to write with a lot of opportunities to get it wrong. By contrast, in Haskell, a function to reverse a list can be defined like this:

```
reverse lst = case lst of
              [] → []
              head:rest → (reverse rest) ++ [head]
```

  The whole function is a single expression that says that if the list is an empty list, you just return the empty list, and if it is not empty then it will have a head followed by the rest of the list, and then you should reverse the rest of the list and concatenate that with a list containing only the head. This conciseness is very useful when synthesizing programs, because it allows you to synthesize non-trivial programs while only having to discover small amounts of code.

  It is important to note that while LLMs are better suited to dealing with verbose languages like Java, the need for conciseness doesn't go away when using LLMs. Longer programs require more resources and bigger attention windows, and there are more opportunities for something to go wrong.

There are long-running debates that sometimes get very religious in nature as to what is the best programming language for this or that purpose. For this course, though, we are not really interested in the question of what language you should use for writing a particular system. What we really care about is the notation that we are going to use when framing a synthesis problem; this notation will often have to be problem specific, but it is very important that this notation be concise and have enough expressiveness to solve our problem, but not much more.

2.4 Representing Programs

In standard programming, programs are represented as strings of text that must follow a grammar, but this representation used to be frowned upon for program synthesis because it is very sparse (most strings are not valid programs), and because it is wasteful to represent programs as strings just to immediately parse those strings into a datastructure. In recent years, though, string representations have gained significant popularity thanks to the success of language models. Today, strings and structured representations are likely to coexist in systems that combine symbolic and neural techniques.

For techniques that predate language models, the preferred representation has traditionally been a datastructure known as an Abstract Syntax Tree (AST), which is just a tree with different kinds of nodes for different kinds of constructs in the language. There is usually a very close correspondence between the structure of the AST and the structure of a parse tree of the program. What makes it *abstract* is that the data-structure can usually ignore information about things like spacing or special characters like brackets, colons and semicolons, since the information they encode can instead by captured by the structure of the AST.

As an example, consider the language of arithmetic expressions. The syntax of such a language can be represented as the following CFG:

$$
\begin{array}{lll}
expr & := & term \mid \\
 & & term + expr \\
term & := & (expr) \mid \\
 & & term * term \\
 & & N
\end{array}
$$

The grammar captures a lot of syntactic information about the language. It describes for example, that in an expression $5 + 3 * 2$, the multiplication takes precedence over the addition, but we can change that by adding parenthesis around $5 + 3$.

An AST, however, can afford to ignore these syntactic details. For this example, we could instead define an AST as a data-structure with three different types of nodes:

```
data AST = Num Int
         | Plus AST AST
         | Times AST AST
```

The block above is Haskell notation that says that an AST type is either a Num, which includes an integer value, or a Plus with two child ASTs or a Times with two child ASTs. If you are unfamiliar with Algebraic datatypes (ADTs) in Haskell, you may find especially useful the discussion of Haskell ADTs in the CIS 1940 course at UPenn, which is available online.)

Note that the distinction between expressions and terms is no longer relevant in this abstract notation; it was only introduced in the grammar for the purpose of disambiguating the parsing of expressions like $5 + (3 * 2)$ and $(5 + 3) * 2$. Instead, this difference is directly reflected in the data-structure. The first would be constructed as `Plus (Num 5) (Times (Num 3) (Num 2))`, while the second would be constructed as `Times (Plus (Num 5) (Num 3)) (Num 2)`.

As mentioned before, synthesis approaches based on deep learning generally prefer to represent programs as strings. In the early days, it was not entirely clear that this was the right approach due to all the aforementioned advantages of structured representations. But string representations made it possible to benefit from the significant engineering efforts being put into neural architectures for natural language processing, and they also made it possible to benefit from pretrained models trained on large amounts of data from the internet, so strings (or more precisely token sequences as we will see in Unit 2) eventually replaced all other representations when it came to working with neural models.

Nevertheless, structured representations remain essential for symbolic approaches, and they are also useful in hybrid approaches that combine symbolic and neural techniques.

2.5 Solving the Invention Challenge in PBE

Let us now return to the problem at hand. Having decided to represent specifications as input-output pairs, we can shift our attention to the Invention challenge. This will require answering two questions: What the space of programs is, and how it is going to be searched.

2.5.1 Defining the Program Space

Having decided on a program representation, the next step is to define the space of programs that the synthesizer is allowed to consider.

Option 1: Domain-Specific Languages Perhaps the most natural way to define a space of programs is to construct a small domain-specific language (DSL), and then set the program space to be the set of all possible programs

that lay within this language. A DSL will typically be defined through a context free grammar (CFG) or directly as an AST datatype, with semantics associated with each element and their composition.

Both the CFG representation and ASTs have the advantage that it is easy to derive all of the possible programs in the language by simply expanding the non-terminal symbols in the grammar, which makes this representation particularly suitable for enumerative search strategies (which we will discuss shortly).

In addition to a context free grammar and a semantics, it is often very useful to have a type system associated with the language. The type system provides an efficient mechanism to rule out programs that while legal with respect to the grammar, are not well-formed with respect to the semantics of the language.

Option 2: Parametric Representations In contrast, constraint-based approaches often rely on parametric representations of the space, where different choices of parameters correspond to different choices for what the program will look like. Parametric representations are more general than grammars; you can usually encode the space represented by a grammar with a parametric representation as long as you are willing to bound the length of programs you want to consider—this is because parametric representations assume the set of parameters is fixed. Sketches, which we will discuss in Lecture 5, are a good example of this kind of representation.

Option 3: Ad-hoc restrictions In some cases, it is necessary to restrict the space of programs in an ad-hoc manner, for example, ruling out programs beyond a certain length, or allowing them to use a particular construct only a few times. This is often done in order to make the search more efficient, or to avoid generating programs that are too complex to be useful. These kinds of ad-hoc restrictions are often used on top of a DSL or a parametric representation, but in principle they can be used directly on top of a general programming language such as Python or Java.

All the options above provide different ways to define a space of programs that is constrained enough to be searched efficiently. When using traditional synthesis techniques, it is relatively easy to focus the synthesis process to consider only programs in a particular space. With neural techniques, however, it can be harder to force the synthesizer to consider only programs within the desired space, but techniques such as constrained decoding and prompt engineering can be effective as we will see in Lecture xx.

2.5.2 Example

As a running example, consider the following language:

$lstExpr$	:=	$sort(lstExpr)$	sorts a list given by lstExpr.
		$lstExpr[intExpr, intExpr]$	selects sub-list from the list given by the start and end position
		$lstExpr + lstExpr$	concatenates two lists
		$recursive(lstExpr)$	calls the program recursively on its argument list; if the list is empty, returns empty without a recursive call
		$[0]$	a list with a single entry containing the number zero
		in	the input list in
$intExpr$	:=	$firstZero(lstExpr)$	position of the first zero in a list
		$len(lstExpr)$	length of a given list
		0	constant zero
		$intExpr + 1$	adds one to a number

In this language, there are two types of expressions, list expressions $lstExpr$, which evaluate to a list, and integer expressions $intExpr$ which evaluate to an integer. Programs in this language have only one input, a list in.

On the one hand, the language is very rich; it includes recursion, concatenation, sorting, search; you can write a ton of interesting programs with this language. For example, the program to reverse a list would be written as follows:

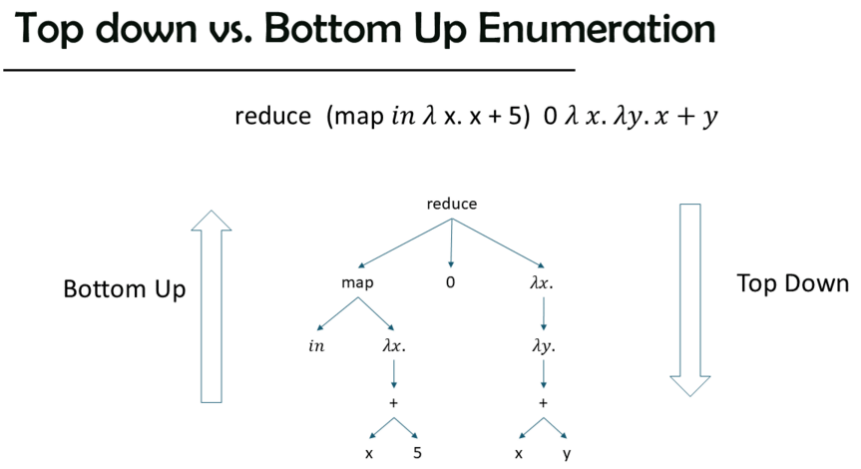
$recursive(in[0 + 1, len(in)]) + in[0, 0]$

Now, consider the following input/output example:

```
in: [1,2,3,4,5,6,7,8]
out: [8,7,6,5,4,3,2,1]
```

If I had the full expressiveness of a general purpose language, say Haskell or Python, there would be an infinite number of programs that could potentially match the example above. But our sample DSL is much more restricted; in fact, it is so restricted that the shortest program matching the example above is actually the correct reversal program. So we can see that the right choice of language can have a significant impact on the our ability to discover programs.

2.5.3 Searching the Program Space



Explicit Enumeration One class of search technique is *Explicit enumeration*. At a high-level, the idea is to explicitly construct different programs until one finds a program that satisfies the observations. In general, though, the space of possible programs that one can generate to satisfy a given specification is too large to enumerate efficiently, so a key aspect of these approaches is how to avoid generating programs that have no hope of satisfying the observations, or which can be shown to be redundant with other programs we have already enumerated. An important distinction in explicit enumeration techniques is whether they are **top down** or **bottom up**. In bottom up enumeration, the idea is to start with low-level components and then discover how to assemble them together into larger programs. By contrast, top-down enumeration starts by trying to discover the high-level structure of the program first, and from there it tries to enumerate the low-level fragments. Essentially, in both cases we are explicitly constructing ASTs, but in one case we are constructing them from the root down, and in the other case we are constructing them from the leafs up.

> $reduce\ (map\ in\ \lambda x.x + 5)\ 0\ (\lambda x.\lambda y.(x + y))$ is a program in functional notation; $\lambda x.x + 5$ just represents a function that takes x as an argument and returns 5. The map function is standard in most programming languages; $map\ lst\ f$ takes a list lst and a function f and constructs a new list by applying f to every element in lst. The reduce function is also quite standard; $reduce\ lst\ init\ f$ assumes that the function f takes two arguments, and then aggregates all the elements in the list using f, so for example, for a list of three elements it would compute $f(lst[2], f(lst[1], f(lst[0], init)))$.

For example, suppose we want to discover a program $reduce\ (map\ in\ \lambda x.x+5)\ 0\ (\lambda x.\lambda y.(x+y))$. In a bottom up search, you start with expressions like $(x+y)$ and $(x+5)$ and build from those expressions to the functions such as $\lambda x.x+5$ and from there you assemble the full program. In contrast, top down search would start with an expression such as $reduce\ \square\ \square\ \square$, and then discover that the first parameter to reduce is $map\ \square\ \square$, and progressively complete the program down to the low-level expressions.

While enumeration may sound a bit brutish, its power should not be underestimated. Often times, it is the simplest techniques that lend themselves to the most efficient implementations, and as Richard Sutton highlighted in the Bitter Lesson, making effective use of computational resources will in time often trump any cleverness in the design of the algorithm. Furthermore, as we will see in the next few lectures, there are many ways to constrain the space which we must enumerate without sacrificing much in terms of simplicity.

Symbolic Search In explicit search, the synthesizer always maintains one or more partially constructed programs that it is currently considering. By contrast, in symbolic search techniques the synthesizer maintains a symbolic representation of the space of all programs that are considered valid. Different symbolic representations lead to different search algorithms. Two of the most popular symbolic representations in use today are Version Space Algebras and Constraint Systems, which we will briefly touch on in Lecture 7.

As an analogy, suppose we want to search for an integer value of n such that $4 * n = 28$. An enumerative search would try all the values one by one until it got to $n = 7$, and then it would declare success. By contrast, a symbolic search technique may perform some algebraic manipulation to deduce that $n = 28/4 = 7$. In this case, symbolic search is clearly better, but even for arithmetic, symbolic manipulation is not always the best choice. Binary search, for example, can be considered a form of explicit search that is actually quite effective in finding solutions to equations that may be too complicated to do algebraic manipulation efficiently.

A Brief Note on Symmetries One important aspect in defining the space of programs which we have so far glossed over is the question of *symmetries*. In program synthesis, we say that a program space has a lot of symmetries if there many different ways of representing the same program. For example, consider the following grammar:

$$\begin{aligned} expr &:= var * N \mid \\ &\quad\ expr; +expr \end{aligned}$$

Now, if we wanted to generate the expression $w * 5 + x * 2 + y * 3 + z * 2$, the grammar above allows us to generate it in many different ways.

$$(w * 5 + x * 2) + (y * 3 + z * 2)$$
$$w * 5 + (x * 2 + (y * 3 + z * 2))$$
$$w * 5 + ((x * 2 + y * 3) + z * 2)$$
$$((w * 5 + x * 2) + y * 3) + z * 2$$
$$\ldots$$

So the grammar above is said to have a lot of symmetries. By contrast, we can define a program space with the grammar below.

$$expr \quad := \quad var * N \mid$$
$$(var * N) + Expr$$

Now, only the second expression in the list above can be generated by this grammar. This grammar in effect forces right associativity of arithmetic expressions, significantly reducing the symmetries in the search space. There are still symmetries due to commutativity of addition, but we have eliminated at least one source of them.

Does this matter? It depends on the search technique and the representation we are using of the search space. Constraint based techniques and some enumerative techniques can be extremely sensitive to symmetries, and will benefit enormously from a representation of the space that eliminates as many of them as possible. On the other hand, there are some techniques that we will study that are mostly oblivous to symmetries.

2.6 A (Bayesian) Probabilistic View of PBE

Bayes theorem

$$P(A \mid B) = \frac{P(B \mid A) * P(A)}{P(B)}$$

$$P(A \cap B) = P(B \mid A) * P(A) = P(A \mid B) * P(B)$$

Bayesian program synthesis

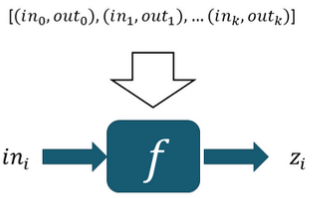
$$P(\textit{prog} \mid \textit{evidence}) = \frac{P(\textit{evidence} \mid \textit{prog}) * P(\textit{prog})}{P(\textit{evidence})}$$

If the evidence is fixed and you just want to find the program with the highest probability

$$P(\textit{prog} \mid \textit{evidence}) \approx P(\textit{evidence} \mid \textit{prog}) * P(\textit{prog})$$

So far, we have assumed that the goal of the synthesizer is to find a program that matches all the given input/output examples, but the problem can also be framed in probabilistic terms. This view has many advantages, from being more tolerant of errors to giving us another avenue for incorporating prior knowledge. The key idea is to use Bayes theorem to derive the probability of a program given some observed evidence. The evidence will generally be input/output examples, but it could also be other observations about the behavior of the program. Bayes theorem allows us to express the desired probability in terms of a prior probability distribution $P(p)$ over the programs, as well as a likelihood function $P(e|p)$ that tells us how likely it is that a program p will produce the evidence e. Using Bayes' rule, we could then obtain the posterior probability of a program given the evidence: $P(p|e)$ (at least up to a normalization constant). Supposing we then had an efficient algorithm for finding the program $p*$ that maximizes the posterior probability, then we would have a solution to the synthesis problem. This formulation can thus be seen as a strict generalization of the original PBE problem, since PBE is equivalent to the case where we take the evidence e to be a set of N input/output examples $\{(in_i, out_i)\}_{i=1}^{N}$ and we define the likelihood function as $p(e \mid p) = 1$ if p for all i produces the output out_i for the input in_i, and 0 otherwise.

Learning from noisy data



Need to trade off quality of f against faithfulness to data

- This requires an error model

$$P(f \mid [(in_i, out_i)]_i) \approx P_f(f) * \prod_i P_{o|z}(out_i \mid f, in_i)$$

Off-by-one Errors

Suppose we know off-by-one errors are possible in the data

- $P_{o|z}(out_i \mid f, in_i) = \begin{cases} 0.5 & f(in_i) = out_i \\ 0.25 & f(in_i) = out_i \pm 1 \\ 0 & else \end{cases}$

If $p(f)$ is uniform, this reduces to

- "Discard programs that are more than one-off on any input"
- "From the remaining programs, select the one that matches the most examples"

Off-by-one Errors

Suppose we know off-by-one errors are possible in the data but others are possible as well.

- $P_{o|z}(out_i \mid f, in_i) = \begin{cases} \frac{1}{z}0.5 & f(in_i) = out_i \\ \frac{1}{z}0.25 & f(in_i) = out_i \pm 1 \\ \epsilon & else \end{cases}$

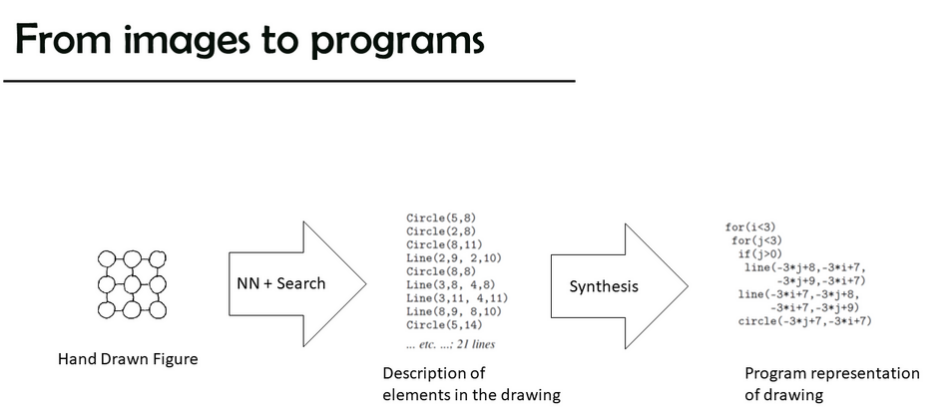
Non-uniform $P(f)$

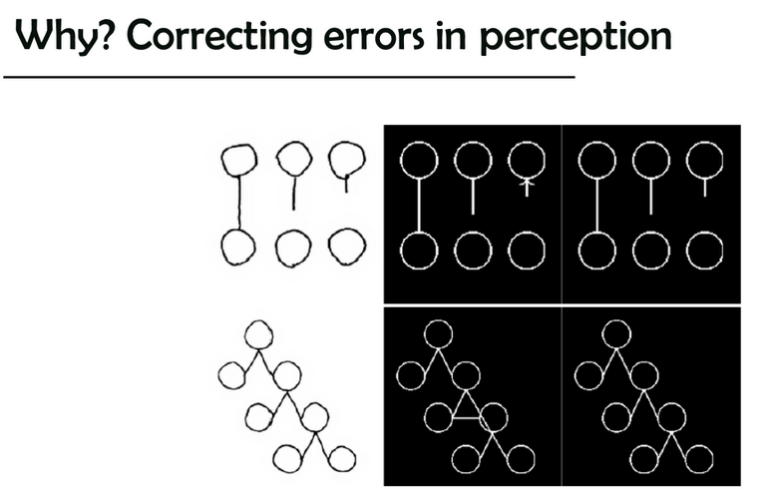
Trade off $P_{o|z}(out_i \mid f, in_i)$ against $P(f)$

- A solution that misses more outputs may still be preferable if it has much higher probability

There are a few advantages to this probabilistic view. The first is that it allows us to deal with the case of noisy evidence, by defining the likelihood function to be non-zero even when the program does not match all the examples. For example, an interesting case to consider is the case where off-by-one errors are possible in the data. The figure illustrates a possible distribution under such an assumption. An interesting observation is that the possibility of errors in the data introduces a necessary tradeoff between the probability of a function and the amount of error it generates.

Ellis et al. [7] explored such a tradeoff between the prior over functions and the output error in their 2018 paper on synthesizing drawing programs from hand-drawn diagrams. This paper uses a neural network to translate a hand-drawn diagram into straight-line code that issues individual drawing commands, and then relies on the Sketch synthesizer to produce a program that is semantically equivalent to the generated sequence of commands. A prior over the space of programs favors programs with loops rather than long sequences of individual drawing commands, and penalizes branches that special-case individual loop iterations.





This prior over programs can be traded off against a simple error model that tolerates small errors in the drawing commands, allowing the synthesizer to make up for small errors in perception. This is illustrated on the second panel of the figure on the right. The first column shows two hand drawings, and the middle column shows the result of running the commands generated by the neural network. The last column shows the result of the most likely program, which trades off accuracy against the result of the neural network in exchange for producing a simpler (and therefore higher likelyhood) program.

A second advantage of this probabilistic view is that it allows us to think about situations analogous to unsupervised learning, where the evidence consists only of output examples, and no input examples are provided. We will not cover this in detail in this course, but the interested reader can refer to the an early work by Ellis, Tenenbaum and Solar-Lezama [8].

Finally, this probabilistic view allows us to bring *learning* into the search problem. For example, we could fit a probability distribution over the grammar, giving us a learnt prior p_θ that we could use to alleviate the issue of underspecification (since the prior tells us which of the correct programs we prefer). We could even have the neural network learn the posterior probability directly as $p_\theta(p \mid e)$, and then use that to guide the search. This is essentially the approach taken by the DeepCoder project of Balog et al. [9], who used a neural network to encode certain features of the specification into a vector, and then decodes it into a vector where each entry corresponds to the probability of each element in the grammar. Such a network can thus then be trained to identify which components are more likely to appear in a program given a particular specification.

However, there is a catch to this framework: Finding $p*$ is intractable for most combinations of the prior and likelihood functions. As mentioned above,

we could learn an approximate posterior $p_\theta(p \mid e)$ and use that to guide the search for a high-likelihood program p, but then we would be forced to inherit all of the limitations of (deep) learning.

References

[1] Patrick H. Winston. Learning structural descriptions from examples. Technical report, Cambridge, MA, USA, 1970.

[2] David Canfield Smith. Pygmalion: A creative programming environment. Technical report, 1976. URL http://worrydream.com/refs/Smith%20-%20Pygmalion.pdf.

[3] Phillip D. Summers. A methodology for lisp program construction from examples. In *Proceedings of the 3rd ACM SIGACT-SIGPLAN Symposium on Principles on Programming Languages*, POPL '76, pages 68–76, New York, NY, USA, 1976. ACM. doi: 10.1145/800168.811541. URL http://doi.acm.org/10.1145/800168.811541.

[4] Phillip D. Summers. A methodology for lisp program construction from examples. *J. ACM*, 24(1):161–175, January 1977. ISSN 0004-5411. doi: 10.1145/321992.322002. URL http://doi.acm.org/10.1145/321992.322002.

[5] Tessa A. Lau and Daniel S. Weld. Programming by demonstration: An inductive learning formulation. In *Proceedings of the 4th International Conference on Intelligent User Interfaces*, IUI '99, pages 145–152, New York, NY, USA, 1999. ACM. ISBN 1-58113-098-8. doi: 10.1145/291080.291104. URL http://doi.acm.org/10.1145/291080.291104.

[6] Tessa Lau. Why programming-by-demonstration systems fail: Lessons learned for usable AI. *AI Magazine*, 30(4):65–67, 2009. URL http://www.aaai.org/ojs/index.php/aimagazine/article/view/2262.

[7] Kevin Ellis, Daniel Ritchie, Armando Solar-Lezama, and Josh Tenenbaum. Learning to infer graphics programs from hand-drawn images. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper/2018/file/6788076842014c83cedadbe6b0ba0314-Paper.pdf.

[8] Kevin Ellis, Armando Solar-Lezama, and Joshua B. Tenenbaum. Unsupervised learning by program synthesis. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, pages 973–981, 2015. URL http://papers.nips.cc/paper/5785-unsupervised-learning-by-program-synthesis.

[9] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL https://openreview.net/forum?id=ByldLrqlx.